№4 ООП

План

- Классы
  - Конструкторы
    - Перегрузка
    - Инициализация
  - This
  - Дружественные объекты
    - Дружественная функция
    - Дружественный метод
    - Дружественный касс
  - Паттерны

Классы

Класс - это *пользовательский тип данных*. Группа объектов, объединённых по общим признакам или имеющих общее поведение.

Классы появились, потому что с ними так же удобно оперировать как и с базовыми типами.

Класс в любом языке описывается по следующей схеме:

```
Начало описание
class <имя>
Интерфейс
     <field> <type>
     <methods> <type>
Реализация
     <field> <type>
```

```
    <methods> <type>
Конец Описания
```

Не стоит *делать переменные публичными*.

Сам *объект должен вести проверку соответствия* свои собственных данных.

У любого класса есть три *модификатора доступа*:

- *public* - доступно всем.
- *protected* - доступно наследникам и самому классу.
- *private* - доступно только классу.

Конструкторы

Конструктор используется для *инициализации* класса *со значением по умолчанию*. Есть несколько типов:

- Вызов конструктора по умолчанию `int x = 3`.
- Вызов специфического конструктора `int x = new int (3)`.
- Явное присвоение `int x; x=3`.
- Конструктор копирования `int y(x)`.
- Конструктор перемещения `move()`.

Перегрузка

```
class Rabbit {
public:
    Rabbit () {}; // Конструктор по умолчанию, компелятор
создаёт его по умолчанию
    Rabbit (int, int); // Так как метод с таким названием
уже существует с другой сигнатурой, то возникает перегрузка
    Rabbit (int, float, char); // Продолжение перегрузки.
}
```

Инициализация

```
class Rabbit {
    int hp;
    int pos;
public:
    Rabbit () {hp = 0}; // Инициализация в теле конструктора. Самый плохой способ инициализации, потому что создаются аллокации
    Rabbit():hp(3), pos(something) {}; // Инициализация через списки инициализации. Если нет указателей.
    Rabbit():hp{3}, pos{something} {}; // Использование юниформы. Для работы с указателями и сложными структурами
    Rabbit(Rabbit& r) // Конструктор копирования
}
```

This

this - это константный указатель на экземпляр текущего класса. Методы отличаются тем, что в них обязательно есть явный или неявный this.

```
void eat(int h) {} // В этой сигнатуре содержится 2 сущности: h и self
```

Дружественные объекты

Дружественная функция

`friend void show(Rabbit)` - прототип сигнатуры. В отличи от методов у функций нет *this*. Это способ нарушить инкапсуляцию. Это только объявление, оно находится внутри класса. Сама реализация может быть в другом файле

Лучше использовать `friend void show(const Rabbit)`, что бы функция не могла поломать объект, а только читать с него.

Дружественный метод

```
class Rabbit {
     int hp;
     friend Iterator::show(const Rabbit)
}
Clsss Itraator {
public:
     show();
}
```

Дружественный касс

```
class Rabbit {
     int hp;
     friend class Iterator
}
Clsss Itraator {
public:
     show();
}
```

Паттерны

```
class UseBd {
static UseBd* instance_connector;
UseBd(){};
UseBd(UseBd&){};
UseBd& opperator = (const UseBd&);

public:
     static UseBd* get_instance(){
     }
}
```

Лучше бы разобраться со `static`.